

ARCHITECTURE AND DESIGN RATIONALE

Serv Audio-to-Orders

How a drive-thru recording becomes signed, structured orders: every part of the sandbox explained simply, why each decision was made, what has been proven, and what is still missing.

24 / 24

eval fixtures passed with real Deepgram + Gemini (34 of 34 conversations found)

100%

item precision and recall, bucket accuracy, status and flags on that run

\$0

spent: Deepgram free credit, Gemini free tier with a hard daily cap

- 1

The problem in plain words
- 2

The core idea: the LLM proposes, code decides
- 3

The system at a glance
- 4

One order, start to finish
- 5

Stage by stage
- 6

Supporting pieces
- 7

The web app
- 8

Testing and evaluation
- 9

Cost and free-tier safety
- 10

Technology choices
- 11

Gaps
- 12

How to explain it
- 13

Open questions for Serv
- A

Commands, glossary, payload fields

1 The problem in plain words

Drive-thru restaurants record the headset audio between the crew and customers. HME is the company whose headset systems make these recordings. Serv wants to turn those recordings into **structured orders**: which car ordered what, in what size, with which changes, and whether anything was unclear.

A recording is messy. One file can hold many cars back to back. Customers change their minds ("actually, make it a Sprite"), ask questions without ordering ("how big is the large?"), split the bill, or drive off. Crew members talk to each other on the same headset ("need fries on two"). There is engine noise, wind and radio.

INPUT AND OUTPUT

Input	One MP3 recording from a drive-thru lane (mono or stereo).
Output	One JSON order per car, sent to Serv's server as a signed HTTP request (a "webhook"), the moment that order is ready.
Also	A web app that shows every step, so a person can check the result against the audio.

What "done" meant (from the brief)

REQUIREMENT	STATUS
<code>pnpm pipeline run <file.mp3></code> produces orders and delivers them to the mock webhook	Done
At least 5 example orders saved in <code>examples/</code> (audio, transcript, order JSON, webhook payload)	Done (7 conversations, 8 orders)
<code>pnpm eval</code> reports accuracy against ground-truth orders	Done (24/24 with real providers)
Next.js app shows a run end to end	Done (Apple Music style UI, PR #1)
README covers setup, how to run, assumptions and known gaps	Done
All 28 edge-case checklist rows have a fixture and pass	Done (28/28)

What this is, and what it is not

This is a **sandbox**: a working, testable prototype that runs on one laptop with free services. Its job is to prove the approach and to make every assumption visible. It has not yet seen a single real HME recording; all audio so far is synthetic (computer voices plus mixed-in noise). Section 11 lists exactly what would change before real use.

2 The core idea: the LLM proposes, code decides

The language model never writes the final order. It only reports what happened in the conversation, as a list of small "events". Plain, deterministic code turns those events into the order.

WHAT THE MODEL OUTPUTS

```
e1 ADD      spicy_chicken
e2 ADD      coke, large
e3 REPLACE  e2 -> sprite
e4 READBACK spicy_chicken, large sprite
           total $7.88
```

WHAT CODE BUILDS FROM IT

```
items:      Spicy Chicken Sandwich  $5.49
           Sprite (large)           $2.39
not_ordered: Coke (replaced by Sprite)
total:      $7.88 (matches the crew)
status:     completed
```

There are 15 event types, each matching something people actually say at a drive-thru:

EVENT	EXAMPLE PHRASE	EVENT	EXAMPLE PHRASE
ADD	"Can I get a #2"	DUPLICATE_LINE	"Same thing for her"
REMOVE	"Scratch the fries"	DECLINE_UPSELL	"No, not the meal"
CHANGE_QTY	"Make that two"	INQUIRE	"How big is the large?"
CHANGE_SIZE	"Actually, large"	OUT_OF_STOCK	"We're out of shakes" (crew)
ADD_MODIFIER / REMOVE_MODIFIER	"No pickles" / "Pickles are fine"	SPLIT_ORDER	"Can we pay separately?"
REPLACE	"Sprite instead of Coke"	CANCEL_ORDER	"Never mind"
SET_COMBO_SLOT	"#2 with a Sprite"	READBACK	"So that's a #2 with a Sprite" (crew)

WHY WE BUILT IT THIS WAY

- **Corrections become natural.** "Coke, actually Sprite" is just two events. The model does not have to do bookkeeping in its head, which is where language models make mistakes.
- **The model cannot invent things.** Prices, item names and combo rules come from the menu file, not from the model. An item ID the menu does not know is caught and routed to human review.
- **It is testable.** The function that builds orders, `replay(events)`, has no network, no clock and no randomness. Feed it the same events and you get the same order every time, so it is covered by unit tests that cost nothing to run.
- **It is auditable.** Every order keeps its event list and a log of how each event was applied, so anyone can see why an item is or is not on the order.
- **It is swappable.** If a better model appears, only the event extractor changes. Everything downstream stays the same.

Two confidence scores, three buckets

Every event carries two separate scores, because there are two separate ways to be unsure:

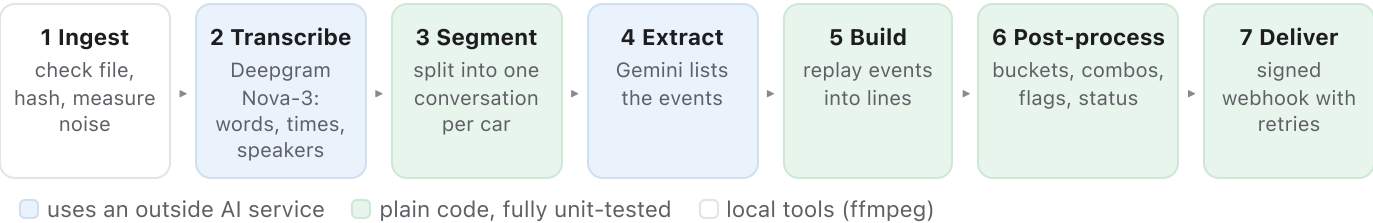
- **Recognition confidence:** are we sure *which* item was said? (Low when the audio was garbled or the word was misheard.) It can never be higher than the speech-to-text system's own confidence in those words.

- **Commitment confidence:** are we sure the customer actually *ordered* it? (Low for "maybe a cookie... nah".)

Those two scores sort every mention into exactly one of three buckets, so nothing the customer said is lost:

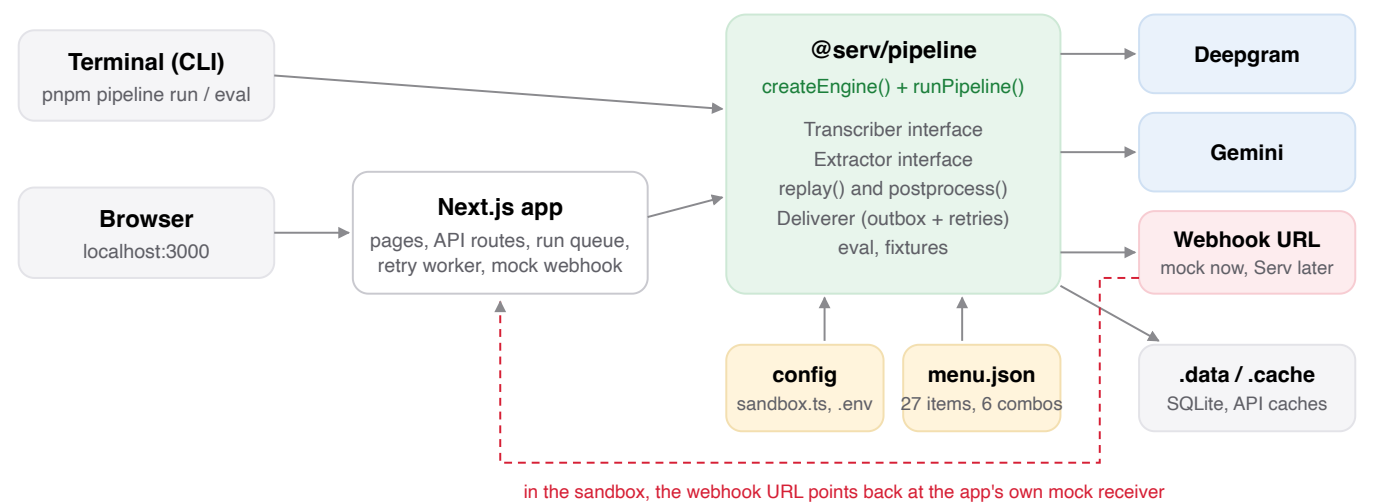
BUCKET	RULE (BOTH THRESHOLDS 0.75 BY DEFAULT)	MEANING FOR SERV
items	Recognition and commitment both high	Ordered. Make it.
needs_review	Committed, but unsure what (includes the top 3 menu candidates)	They ordered something; a person should check.
not_ordered	Cancelled, replaced, declined upsell, out of stock, only asked about, or hesitated	Not made, but useful signal (upsell and demand data).

3 The system at a glance



Stage 3 can also ask Gemini a yes/no question for gaps the rules cannot decide, and stage 2 can ask it which voice is the crew. Both are optional tie-breakers, and both fall back to rules when Gemini is unavailable.

How the pieces fit together



The project is one repository holding two programs that share the same engine:

- **The pipeline package** (`packages/pipeline`) holds all the logic. It is used directly from the terminal (`pnpm pipeline run` , `pnpm eval`).

- **The web app** (apps/web) imports that same package. Its run queue calls the exact same `runPipeline()` the terminal uses, so the UI can never drift from the real behavior. The browser even runs the same `replay()` function to rebuild an order live as the audio plays.

Repository layout

```
packages/pipeline/src/
  ingest/      file checks, SHA-256 hash, noise levels, recording start time
  transcribe/  Transcriber interface; Deepgram adapter; speaker roles; ground-truth transcriber
  segment/     phrase cues, gap scoring, LLM tie-breaker
  extract/     Extractor interface; Gemini adapter, prompt, schema, validation; fuzzy and oracle
  build/       replay(events) -> order state (pure)
  postprocess/ buckets, combo detection, flags, status (pure)
  webhook/     payload, Standard Webhooks signing, outbox + retries, mock receiver logic
  store/       SQLite tables (runs, orders, outbox, attempts, mock inbox)
  eval/        scoring, edge-case checklist, webhook self-check
  fixtures/    synthetic audio generator
  cli.ts       run | eval | resend | worker | examples | settings
apps/web/      Next.js pages, API routes, /api/mock-webhook
config/        sandbox.ts: every Serv-dependent value, with placeholder tracking
menu/menu.json "Sandbox Burger" catalog
fixtures/      22 scripted conversations + 2 multi-car compilations, audio, timelines
examples/      deliverable example orders
eval/report.json latest eval result
```

WHY ONE TYPESCRIPT MONOREPO

One language end to end means the order schema, the validation rules and the builder are written once and used by the CLI, the server and the browser. A pnpm workspace keeps the packages separate (logic, config, UI) while letting them import each other directly.

4 One order, start to finish

This is a real run from `examples/04_correction`: the mono headset fixture, processed by real Deepgram and Gemini. It is the clearest way to explain the system to someone new.

Step 1 and 2: the audio becomes a transcript

Ingest checks the 25-second MP3 and hashes it. Deepgram returns words with timestamps and confidence. On this mono file, Deepgram's speaker separation heard only one voice, so the pipeline decided who was speaking from the wording itself (Section 5.2 explains why). That is why every role below is marked as a guess.

u1	crew?	Hi. Welcome to Sandbox Burger. Go ahead whenever you're ready.
u2	customer?	Can I get a spicy chicken sandwich and a large Coke?
u3	customer?	Actually,
u4	customer?	make that a Sprite instead of the Coke.
u5	crew?	Spicy Chicken Sandwich and a large Sprite.
u6	crew?	Anything else?
u7	customer?	Nope.
u8	crew?	Your total is \$7.88.
u9	crew?	See you at the window.

Step 3: segmentation

"Welcome to" is a start cue, and "your total is" and "see you at the window" are end cues. No gap inside scores as a new car, so this is one conversation, `seg_1`, from 0.88s to 24.6s. It has a greeting and a closing, so it is neither truncated nor abandoned.

Step 4: Gemini proposes events

```
e1 ADD      spicy_chicken      from u2  recognition 0.99  commitment 1.0
e2 ADD      coke, size large    from u2  recognition 0.99  commitment 1.0
e3 REPLACE  target e2 -> sprite from u4  recognition 0.90  commitment 1.0
e4 READBACK spicy_chicken; sprite large; amount $7.88 from u5, u8
```

Validation checks every ID against the menu, drops any utterance ID that does not exist, and caps recognition at Deepgram's word confidence (u4 was heard at 0.896, so e3 is capped at 0.90).

Step 5: replay builds the lines

```
e1 ADD      applied  added Spicy Chicken Sandwich x1
e2 ADD      applied  added Coke x1
e3 REPLACE  applied  e2 replaced by e3          (Coke line kept, marked "replaced")
e4 READBACK applied  readback matches          (no readback_mismatch flag)
```

Note that the large size moved from the Coke to the Sprite automatically. That is a rule in code, not something the model had to remember.

Step 6: post-processing

Both remaining lines are above 0.75 on both scores, so they land in `items`. The Coke lands in `not_ordered` with reason `replaced` and `replaced_by: sprite`. Prices come from the menu: $\$5.49 + \$2.39 = \$7.88$, which matches what the crew said, so there is no `total_mismatch`. Status is `completed`. The only flag is `placeholder_values`, which every sandbox order carries until Serv supplies real IDs and secrets.

Step 7: delivery

The order is written to the outbox and POSTed immediately with these headers:

```
webhook-id:      ord_01M443TADNRGY39TYRPRVV8ZH9_v1      (same on every retry)
webhook-timestamp: 1791139588
webhook-signature: v1,9E1uULRRDQGP1S9pDFA91w3qT31G79Y0HfbIr8fB4FM=
x-delivery-attempt: 1
user-agent:      serv-audio-orders/0.1
```

The mock receiver checks the signature, checks that the timestamp is less than 5 minutes old, stores the payload, and returns 200. If it had returned 500, the sender would retry with the identical body and ID, and the receiver would drop any duplicate.

5 Stage by stage

Each stage below has three parts: what it does in one sentence, how it works, and why it was built that way.

5.1 Ingest

In one sentence: make sure the file is real audio worth processing, and gather facts about it.

- **Probe** with `ffprobe`: codec, sample rate, channels, duration.
- **Reject** files that are missing, empty, corrupt, shorter than 0.5 seconds, or silent (loudest point below -60 dBFS). A rejected file still gets a failed run record with a clear reason.
- **Hash** the file with SHA-256. The hash is the cache key for transcription, so the same file is never paid for twice.
- **Measure loudness** every 100 ms. Later, each conversation's signal-to-noise ratio is estimated as the gap between its loud moments (speech, 90th percentile) and its quiet moments (noise floor, 10th percentile).
- **Find the wall-clock start** of the recording: `AUDIO_START_UTC` if set, else a timestamp in the filename (`20261003T184000Z` or `2026-10-03_18-40-00`), else the file's modified time. Which one was used is recorded as `timestamp_source`.

WHY

Failing fast on bad input avoids wasting paid transcription minutes. Hashing makes reruns free. The loudness measurement exists because of a finding during tuning: speech-to-text confidence stays high on loud, steady noise (engine idle), so confidence alone could not flag the heavy-noise fixture. Measured SNR was about 11 dB on heavy noise, 21 to 22 dB on moderate, and 29 dB or more on clean, so the flag threshold sits at 15 dB. Wall-clock time matters because Serv will match orders to cars and payments by time.

5.2 Transcribe

In one sentence: turn speech into time-stamped words, and label each line as crew or customer.

The pipeline calls **Deepgram Nova-3** with punctuation, smart formatting, utterance splitting at 0.8 seconds of pause, filler words kept ("uh", "um"), and every menu item name sent as a "keyterm" so the model leans toward menu vocabulary. Words below 0.6 confidence are marked `low_conf` and carried forward. The raw

response is saved under `.cache/<hash>/`.

Deepgram returns speaker numbers, not roles. Deciding who is crew uses one of three methods, in order of preference:

METHOD	WHEN	HOW
Channel	Stereo audio and <code>CHANNEL_MAP</code> is set (e.g. <code>0=customer,1=crew</code>)	Each channel is transcribed separately; the role comes from the channel. Most reliable.
Diarization	Mono, and Deepgram hears distinct voices	The voice that greets, asks "anything else" or reads a total is crew. A voice that only says kitchen chatter is crew too. Ties go to a one-line Gemini question, then to "the first voice is the crew greeting".
Wording	Diarization put 90% or more of the lines under one voice	Each line is scored with crew phrases ("anything else", totals, "okay, large Sprite") and customer phrases ("can I get", "that's it", "never mind"). Undecided lines follow turn-taking: a line that continues an unfinished one keeps the speaker; otherwise the speaker alternates. These lines are marked as guesses.

WHY DEEPGRAM NOVA-3

It returns word-level timestamps and confidence (needed to sync the UI and to cap recognition), supports diarization and multichannel, supports keyterm prompting for menu words, and comes with \$200 of free credit. It sits behind a `Transcriber` interface, so a different provider would be one new file.

WHY ROLE INFERENCE FROM WORDING

This was the single biggest accuracy fix. The first real-provider eval scored 12 of 24. The cause: on mono headset audio, Deepgram's diarization heard everyone as one speaker, so every line was labeled crew, and the model was told to ignore crew lines when adding items. Wording-based roles were right on 229 of 234 fixture lines (98%), against 174 of 234 (74%) from diarization. Marking these lines as guesses (`[customer?]` in the prompt) lets the model overrule a wrong label from context. Caveat: the phrase lists were tuned on these same fixtures.

WHY LANGUAGE "MULTI" (A DELIBERATE CHANGE FROM THE BRIEF)

The brief said `language=en`. Nova-3's multilingual mode handles English/Spanish code-switching, still supports keyterms, and tags each word with its language, which gives the `non_english` flag without a second pass. `STT_LANGUAGE=en` switches back.

5.3 Segment

In one sentence: cut one long recording into one conversation per car.

Every gap between two lines gets a score from 0 (same car) to 1 (new car):

SIGNAL	EFFECT ON SCORE
--------	-----------------

Crew end cue before the gap ("your total is", "pull forward", "see you at the window")	+0.40 (+0.30 if a short customer "thanks" follows it)
Crew start cue after the gap ("welcome to", "order when you're ready")	+0.45
Silence of 8s or more (<code>SEGMENT_GAP_S</code>); 16s or more	+0.35; +0.45. Shorter pauses add a little; under 1.5s subtracts 0.15
Restart cue ("start over", "say that again")	-0.50 (keeps the order together)

A score of 0.7 or more is a boundary. Below 0.35 is the same order. In between, Gemini is asked "does a new customer start here?" with three lines on either side. No conversation may exceed 6 minutes; an overlong one is split at its best-scoring gap. Each conversation then gets its facts: greeting and closing found or not, truncated at the start or end of the file, mean word confidence, the share of non-English customer lines, possible crosstalk (overlapping speakers), and which lines are crew-to-crew chatter (excluded from extraction).

WHY RULES FIRST, LLM ONLY FOR TIES

Most boundaries are obvious ("see you at the window", silence, "welcome to"), and rules are free, instant and predictable. The LLM is only consulted in the gray zone, which keeps cost near zero and keeps segmentation explainable: every boundary in the UI shows its score, its signals, and whether rules, the LLM or the 6-minute cap decided it. On ground truth and on the real-provider run, segmentation found 34 of 34 conversations with no extras; mean boundary error on the real run was 0.13s at the start and 0.38s at the end.

5.4 Extract

In one sentence: ask Gemini to list the events in one conversation, then check its answer.

- **One call per conversation**, temperature 0, with a compact menu, the conversation's lines (IDs, roles, guessed roles marked with "?", unclear words listed), and 11 rules. Examples of rules: only the customer adds items; a crew suggestion counts only if accepted; questions are `INQUIRE` ; use menu IDs only, or `null` plus the customer's words if unsure.
- **Structured output:** Gemini must return JSON matching a schema generated from the same zod types the code uses.
- **Validation in code:** unknown item IDs are fuzzy-matched to the menu (accepted at 0.75 similarity or above); still-unknown IDs trigger one repair request; anything left becomes a `needs_review` line with candidates. Unknown modifiers are dropped with a warning. Recognition is capped at the ASR's mean word confidence.
- **Fallback:** if Gemini is out of budget or keeps failing, a keyword matcher runs instead and puts everything in `needs_review` . The run still completes; nothing is silently guessed.
- **Safety and logging:** safety settings are pinned on every request, a blocked response raises a typed error (and the run degrades to the keyword fallback with a warning), each request has a 90-second deadline, and each request writes one structured `llm_call` log line.

- **Cache:** every response is stored in `.cache/llm/`, keyed by model, prompt and input, so reruns of the same audio cost nothing.

Three extractors exist behind the `Extractor` interface: **gemini** (the real one), **fuzzy** (keyword fallback), and **oracle** (replays the hand-written events from the fixture script; used to measure the pipeline's ceiling, see Section 8).

WHY GEMINI 3.5 FLASH-LITE

The brief required a free tier. Gemini's free API tier supports structured JSON output. The newer "Flash latest" model was tried first, but Google's free tier allows it only 20 requests per day, and one full eval needs about 34. Flash-Lite fits a full eval inside the free tier and reached 24/24. `GEMINI_MODEL` switches models without code changes.

WHY ONE CALL PER CONVERSATION (NOT PER LINE, NOT PER FILE)

A per-line call cannot see that "make that two" refers to an earlier line. A whole-file call mixes cars together and grows with file length. One conversation is the natural unit: it holds all the context an order needs and nothing from other cars. It averaged about 3,000 tokens per conversation (102k tokens for 34 conversations).

HOW THE PROMPT GOT TO 24/24

Four prompt versions, each tested on the full eval: v1 scored 12/24; v2 with wording-based roles scored 19/24; v3 scored 22/24; v4 scored 24/24. Changes included treating "?" labels as guesses, making item questions `INQUIRE` with an ID, separating "unsure what" from "unsure whether", and pointing "same thing for her" at the most recent line. Code changes in the builder and post-processing fixed the rest (split payments by spoken totals, hesitation routing, unidentified items always going to review).

5.5 Build: `replay()`

In one sentence: apply the events in time order to a list of order lines, like a bank applying transactions to a balance.

- Each line is named after the event that created it, so "the line from e2" is unambiguous. If a line was replaced, references to it follow the replacement chain.
- Removed, replaced, cancelled and out-of-stock lines are never deleted; they are kept with a reason, so they can appear in `not_ordered`.
- "Two burgers, one with no pickles" splits into two lines of one, one with `no_pickles`.
- Combo modifiers go on the component they apply to (no pickles goes on the burger inside the meal, not on the drink).
- Each crew readback is compared to the order at that moment; differences raise `readback_mismatch`.
- "Can we pay separately?" creates a second order in the same car, sharing a `group_id`. If it is asked before anything is ordered, each spoken total closes one order.
- Removing something the customer only hesitated about counts as "uncommitted", not "cancelled".

Every event produces a log entry (applied or not, and why), which the UI shows next to the order.

5.6 Post-process

In one sentence: sort lines into buckets, price them, look for missed meal deals, and set flags and status.

- **Buckets** as in Section 2. An item nobody could identify always goes to `needs_review`, even if the model marked low commitment, because models tend to confuse "unsure what" with "unsure whether", and a person should look either way.
- **Combo opportunities:** if separate lines would fill every slot of a meal for less money (burger + fries + drink), the order gets a `combo_opportunity` with the savings. It is never auto-converted. If the customer explicitly declined the meal, `customer_declined_combo` is set too.
- **Totals:** computed from menu prices (with `TAX_RATE`, currently 0) and compared to the total the crew spoke, within 5 cents.
- **Status**, first match wins: cancelled, then incomplete (truncated audio), then abandoned (no closing), then `needs_review` (anything to review, or readback mismatch), then completed.

FLAG	RAISED WHEN	FLAG	RAISED WHEN
<code>needs_review_present</code>	any <code>needs_review</code> line	<code>truncated_start</code> / <code>_end</code>	file starts or ends mid-order
<code>readback_mismatch</code>	crew read back something different	<code>split_order</code>	separate payments in one car
<code>total_mismatch</code>	spoken total differs by more than \$0.05	<code>non_english</code>	30%+ of customer lines non-English, or the model says so
<code>missing_required_slot</code>	a meal has no drink chosen	<code>low_audio_quality</code>	mean word confidence < 0.8 or SNR < 15 dB
<code>combo_opportunity</code>	separate items would be cheaper as a meal	<code>crosstalk_suspected</code>	speakers overlap by more than 0.5s
		<code>placeholder_values</code>	any Serv setting is still a sandbox default

WHY NEVER AUTO-CONVERT TO A COMBO

Changing what a customer ordered is a business decision, not a parsing decision. The pipeline reports the opportunity and the savings; Serv decides what to do with it.

5.7 Deliver

In one sentence: send each order to Serv's server reliably, securely, and exactly once from the receiver's point of view.

1. **Outbox first.** The order is written to an SQLite "outbox" table before anything is sent, then sent immediately.
2. **Signed.** Headers follow the Standard Webhooks spec (the scheme used by Svix and others): an HMAC-SHA256 signature over `id.timestamp.body` with a `whsec_` secret. The implementation is verified against the spec's reference test vector.
3. **Success** is any 2xx within 10 seconds.

4. **Retry** on network errors, timeouts, 408, 429 and 5xx, honoring `Retry-After`. Other 4xx responses mean "your request is wrong", so they are marked failed with the response body.
5. **Fast phase** retries at about 1, 2, 4, 8, 16, 32 seconds (randomized "full jitter"). **Slow phase** at 5 min, 30 min, 2 h, 5 h, 10 h, 10 h. After 13 attempts over about 28 hours: dead letter, with a Resend button in the UI and `pnpm pipeline resend <order_id>`.

WHY EACH PIECE

- **Outbox:** if the process crashes mid-send, the order is still on disk and the worker picks it up. Rows stuck "delivering" after a crash are reset on startup.
- **Constant webhook-id:** retries carry the same ID and the same body, so the receiver can drop duplicates. That makes "at least once" delivery behave like "exactly once".
- **Timestamp in the signature:** the receiver rejects anything older than 5 minutes, so a captured request cannot be replayed later.
- **Standard Webhooks:** Serv does not need custom code to verify; any Standard Webhooks library works. For their likely AWS setup (API Gateway in front of a Lambda), only `WEBHOOK_URL` and `WEBHOOK_SECRET` change.
- **Two retry phases:** cars move in minutes, so early retries must be fast. Real outages last hours, so later retries must be patient. Jitter stops many retries from hitting a recovering server at the same instant.

6 Supporting pieces

The menu catalog (`menu/menu.json`)

The single source of truth for what can be ordered: a generic chain called "Sandbox Burger" with 27 items, 6 numbered meals and 29 modifiers. Each item has an ID, name, aliases (common mishearings and nicknames), category, sizes and prices. Each meal has slots: a fixed entree, a default side that can be swapped, and a required drink. It deliberately includes an item that can run out (shakes), a free item (water), and a confusingly named item ("Sandy Fluffle") to exercise fuzzy matching.

WHY A CATALOG AT ALL

Constraining the model to catalog IDs is what makes outputs checkable. Prices let the pipeline compare its total with the crew's spoken total, a cheap and strong accuracy check that needs no ground truth.

Configuration and placeholders (`config/sandbox.ts`)

Every value that depends on Serv lives in one file. Any value still at its sandbox default is a **placeholder**: it shows a yellow badge in the UI, is listed by `pnpm pipeline settings`, and adds the `placeholder_values` flag to every order.

SETTING	SANDBOX VALUE	REPLACE WHEN
LOCATION_ID , LANE_ID	store_demo_001 , lane_1	Serv shares site and lane IDs

WEBHOOK_URL	the app's own <code>/api/mock-webhook</code>	Serv shares an endpoint
WEBHOOK_SECRET	generated dev secret in <code>.data/</code>	Serv issues one
CHANNEL_MAP	unset (diarization or wording)	an HME sample confirms channels
AUDIO_START_UTC	unset (filename, then file time)	HME metadata format is known
Menu	<code>sandbox-1</code>	Serv names a brand
TAX_RATE	0	the site is known
Bucket thresholds	0.75 / 0.75	real audio with actual orders exists

WHY

Assumptions hide in code. Putting them in one place, and making them visible on every screen and every order, means nobody can mistake a sandbox result for a production one, and the list of "things Serv must tell us" is always current.

Secrets

API keys and the webhook secret live only in environment variables (`.env` , which git ignores). The repo ships `.env.example` with placeholders only. The UI shows only the first 10 characters of the webhook secret.

Storage (`.data/sandbox.db` , SQLite)

TABLE	HOLDS
<code>runs</code>	one row per processed file: status, stage, audio info, transcript, segmentation, usage, timings, errors
<code>orders</code>	each order version: payload, events, raw model output, build log
<code>outbox</code>	one row per webhook to send: body, status (pending, delivering, delivered, failed, dead), next attempt time
<code>attempts</code>	every HTTP attempt: time, phase, status code, latency, response body, Retry-After
<code>mock_inbox</code> , <code>mock_settings</code>	what the mock receiver got, its signature check, and its failure mode

WHY SQLITE

Zero setup, one file, real transactions, and fast enough for one machine. The outbox pattern needs a durable store, and SQLite gives that without running a database server. It is the first thing to replace for a deployment (Section 11).

Caches (`.cache/`)

Raw Deepgram responses (keyed by file hash, mode and language) and Gemini responses (keyed by model, thinking level, prompt and input). They make reruns free and reproducible, which is why the prompt could be tuned four times on a free budget. Changing the prompt changes the key, so a new prompt costs

one fresh request per conversation.

7 The web app

A Next.js app at `localhost:3000` that makes every step visible. Started with `pnpm dev`.

PAGE	WHAT IT SHOWS
Runs /	A shelf of fixture "albums" to pick from (or upload an MP3), options (transcriber, extractor, speaker roles, send webhook), past runs, and the settings panel with placeholder badges.
Run /runs/[id]	A player bar; the waveform with each conversation shaded; a speaker lane; the transcript as "lyrics" (customer left, crew right, guessed roles marked, low-confidence words underlined, chatter struck through; click a line to seek); each order's three buckets, flags, combo opportunities, event log and raw model output; and every webhook attempt with a payload viewer and Resend. With "Live replay" on, the order rebuilds itself as the audio plays, using the same <code>replay()</code> in the browser.
Mock Webhook	Received payloads, signature result, duplicates, and switches to make it return 500, 429 with Retry-After, or hang, to demonstrate retries.
Eval	The latest <code>eval/report.json</code> : headline metrics, the 28-row checklist, and per-fixture differences.

How a run executes in the app

1. The form POSTs to `/api/runs`. Uploads (up to 100 MB) are saved under `.data/uploads/`.
2. A run row is created as "queued" and the job goes into an in-memory queue that processes one run at a time.
3. The page polls the run's status and redraws as stages complete.
4. The slow-phase webhook worker runs inside the Next.js server (started from `instrumentation.ts`) and checks for due retries every 5 seconds.

Design and UI library

The visual design follows the Apple Music desktop app: sidebar navigation, "album" artwork generated from each fixture's name, a track-list for orders, a lyrics-style transcript, and a player bar. The system font (`-apple-system`, SF Pro) is used throughout, and light and dark mode follow the operating system.

Components come from **shadcn/ui** (buttons, selects, switches, tables, tooltips, sliders, collapsibles) built on Radix primitives and Tailwind CSS. *Note: this UI library migration is finished and checked visually, but not yet committed; see Section 11.*

WHY THESE CHOICES

- **Next.js:** server routes and pages in one app, and it can import the pipeline package directly, so the UI runs the real code rather than a copy.
- **wavesurfer.js:** a mature waveform player with region shading, which is what syncing audio to transcript and orders needs.
- **shadcn/ui:** the components are copied into the repo as source, so there is no runtime library to lock in, every component can be restyled, and Radix provides keyboard and screen-reader behavior for free.
- **In-process queue:** a sandbox on one machine does not need Redis or a job service. The trade-off is listed in Section 11.
- **Apple Music style:** the run page is literally "play a recording and watch what it produced", so a media-player layout makes the demo self-explanatory for Serv.

8 Testing and evaluation

Synthetic test data

No real HME audio was available, so the sandbox generates its own with known correct answers:

- **22 scripted conversations** in `fixtures/scripts/`. Each has turns (speaker, text, pause), hand-written events, and an expected result (items, needs review, not ordered, flags, status, group).
- **Audio rendering** with macOS's built-in voices (free and local): distinct voices for crew and customers, plus a Spanish voice.
- **Two layouts:** stereo (customer left, crew right, 16 kHz) and a mono headset mix (band-limited to 8 kHz).
- **Noise:** synthetic engine idle, wind and car radio at clean, moderate or heavy levels.
- **2 compilations** stitch several scripts into one long multi-car file to test segmentation.
- Each audio file gets a `.timeline.json` with exact line and order times, which also powers the free "ground truth" transcriber.

Every one of the 28 rows in the brief's edge-case checklist has a fixture (rows 27 and 28, webhook failure and rate limiting, are tested by a self-check against a throwaway local receiver).

What the eval measures

METRIC	DEFINITION
Segmentation	Missed and extra conversations; start and end error in seconds
Item precision / recall	Exact match on menu ID, quantity, size, modifiers and meal components
Bucket accuracy	Each mention in the right bucket with the right reason
Status and flags	Exact match (ignoring <code>placeholder_values</code>)
Checklist	Pass or fail for each of the 28 edge cases

Results

CONFIGURATION	PASSED	ITEM P / R	BUCKETS	SEGMENTS
Ground-truth transcript + oracle events (the ceiling)	24/24	100% / 100%	100%	34/34
Ground-truth transcript + keyword fallback	0/24	100% / 0%	1%	34/34
Deepgram Nova-3 + Gemini 3.5 Flash-Lite (mono, prompt v4)	24/24	100% / 100%	100%	34/34

WHY THREE CONFIGURATIONS

- **The ceiling** feeds the pipeline perfect transcripts and perfect events. If it fails, the bug is in our code (segmentation, builder, scoring), not in a model. It costs nothing to run.
- **The keyword fallback** shows the safety net works as designed: it finds nothing confidently, so it sends everything to review rather than guessing.
- **The real providers** measure the system as it would actually run. The gap between this row and the ceiling is the cost of imperfect speech recognition and an imperfect model.

Accuracy progression with real providers

RUN	PASSED	ITEM P / R	MAIN CHANGE
Prompt v1	12/24	93.5% / 80.6%	Baseline. Diarization heard one voice; every line was "crew".
Prompt v2 + wording roles	19/24	100% / 97.2%	Per-line role inference; guessed labels marked
Prompt v3	22/24	98.6% / 98.6%	Inquiry and commitment rules; builder fixes for splits and hesitation
Prompt v4	24/24	100% / 100%	"Same thing for her" targets the latest line; SNR-based noise flag; language from the model

READ THESE NUMBERS HONESTLY

- The fixtures that measured accuracy are the same fixtures that guided the fixes. There is no held-out test set, so 24/24 shows the approach can work, not that it generalizes.
- It was one run at temperature 0. Some run-to-run variation is expected.
- Only the mono layout was evaluated with real providers. The stereo channel path has never been sent to Deepgram.
- The audio is computer voices with synthetic noise. Real drive-thru audio is harder.

Unit tests

107 tests, all passing, with no network calls, colocated next to the code they test. 97 cover the pipeline: the builder on every checklist scenario, replay edge cases, segmentation, role inference and noise estimates, webhook signing (including the Standard Webhooks reference vector), delivery and retries with a fake

clock, the retry helper, Gemini safety-block detection, and the Gemini budget guard. 10 cover the web API routes' success and error responses. A Playwright test drives the golden path in a real browser: pick a fixture, run it with the free providers, see the order and the delivered webhook.

Code standards

ESLint, `tsc --noEmit` , Vitest and Playwright are the gates, and a GitHub Actions workflow runs them on pull requests. The runtime is Node 22. External calls use a shared retry helper with exponential backoff, and every API route returns the same error shape. Every Gemini call pins its safety settings, detects safety blocks, has a per-request timeout and writes one `llm_call` log line. Gemini uses an AI Studio key rather than Vertex AI, because Vertex has no free tier.

9 Cost and free-tier safety

The brief said "free tiers only". Both providers are used only within free allowances, and usage is logged on every run.

Gemini: how billing is prevented

- 1. **The real guarantee is on Google's side.** Google bills only if the Cloud project behind the API key has a billing account. The key was created in AI Studio on a project with no billing account; Google's own 429 response confirmed it as `FreeTier` . Without billing, exceeding a limit returns an error, never a charge.
- 2. **A hard daily cap** (`GEMINI_DAILY_CAP` , default 200 requests per Pacific-time day, matching Google's quota day), counted in a ledger file before every request, retries included. One ledger per model, because Google's quotas are per model.
- 3. **Daily-quota errors stop immediately** instead of retrying, and mark the day as used up.
- 4. **Graceful fallback:** out of budget means the keyword extractor and rule-based segmentation take over; the run still completes.
- 5. **Small outputs:** "thinking" set to low cut output tokens about five-fold on a test conversation (3,373 to 646) with the same result.
- 6. **Pacing:** 10 requests per minute by default (`GEMINI_RPM`).
- 7. **Caching:** identical requests are never sent twice.

Deepgram

Deepgram's \$200 sign-up credit covers the sandbox. The first full eval used about 15.8 audio minutes; every later run used 0 new minutes because responses are cached by file hash. Minutes billed are logged per run.

What a run costs in practice

ITEM	MEASURED
Gemini tokens per conversation	about 3,000 (79,870 in + 22,329 out for 34 conversations)

Gemini requests per full eval	34 (about one per conversation, plus occasional tie-break questions)
Deepgram minutes per full eval (first time)	about 15.8
Cost of the ceiling eval, unit tests, examples from cache	\$0, 0 requests

Data-use note. On Gemini's free tier, Google may use prompts and responses to improve its products. That is fine for synthetic fixtures. Real customer transcripts should not be sent until Serv approves the data handling, which may mean a paid tier or another provider with a no-training guarantee.

10 Technology choices

CONCERN	CHOICE	WHY
Language	TypeScript (strict), Node 20+	One language for pipeline, server and browser; strict types catch schema drift between stages.
Repo	pnpm workspace monorepo	Separate packages that import each other directly; fast, disk-efficient installs.
Speech to text	Deepgram Nova-3	Word timestamps and confidence, diarization, multichannel, keyterms, multilingual; \$200 free credit.
LLM	Gemini 3.5 Flash-Lite via <code>@google/genai</code>	Free tier large enough for a full eval; structured JSON output; behind an interface.
Schemas	zod	One definition gives TypeScript types, runtime validation, and the JSON Schema sent to the LLM.
Fuzzy matching	fuse.js	Maps misheard names and aliases to menu IDs; small and dependency-free.
Audio tools	ffmpeg-static, ffprobe-static	Probing, loudness, channel handling and fixture generation, bundled so nobody installs ffmpeg by hand.
Storage	SQLite via better-sqlite3	Durable outbox and run history with zero setup.
Webhook signing	Standard Webhooks (HMAC-SHA256)	An open spec with libraries in every language; replay protection and dedupe built in.
Tests and lint	Vitest, Playwright, ESLint	Fast, TypeScript-native; the core logic is pure functions that need no mocks.
UI	Next.js 16, React 19, Tailwind v4, shadcn/ui, wavesurfer.js	Server routes plus pages in one app; owned, accessible components; waveform with regions.
Test audio	macOS <code>say</code> voices	Free and local; reproducible fixtures without spending TTS credit.

Every provider call sits behind an interface (`Transcriber` , `Extractor` , plus judge interfaces for the tie-breakers), per the brief, so swapping Deepgram or Gemini is one new adapter file and a config value.

11 Gaps

Ranked by how much they matter before real use. High blocks a pilot on real data. Medium needed before production. Low housekeeping or polish.

Accuracy and data

GAP		DETAIL AND SUGGESTED NEXT STEP
High	No real HME audio tested	All results are on computer voices with synthetic noise. Next: get 20 to 50 real recordings with the orders the POS actually rang up, add them as fixtures, and re-run the eval.
High	No held-out test set	The same 24 fixtures guided the prompt and phrase-list tuning and measured the result. Next: freeze a held-out set (real audio preferably) that is never used for tuning.
High	Stereo channel path never run live	Every Deepgram call so far was mono with diarization. The multichannel path is unit-tested but has never hit the real API, and the stereo layout was not evaluated. Next: one eval run with <code>--layout stereo</code> (about 16 Deepgram minutes) and confirm what channels HME actually records.
Medium	Thresholds are guesses	Bucket thresholds (0.75), low-confidence word (0.6), low audio quality (0.8 confidence, 15 dB SNR) were set on synthetic audio. Next: calibrate on real labeled audio.
Medium	Role phrases are English-centric and fixture-tuned	The wording-based role inference uses about 30 phrase patterns (a few Spanish). Real crews use their own scripts. Next: measure role accuracy on real audio; consider using the LLM for roles when diarization collapses.
Medium	Single eval run	Run-to-run variance at temperature 0 is unmeasured. Next: run the eval three times when budget allows and report the spread.
Medium	Tax not modeled	<code>TAX_RATE=0</code> . Real spoken totals include tax, so <code>total_mismatch</code> would fire on most real orders until the rate is set per site.
Low	Generic menu	Real menus have more items, regional names, promos and limited-time offers. The keyterm list is capped by Deepgram at 500 tokens, so only item names (not aliases) are sent.
Low	Combo search is greedy	Combo opportunities are found meal by meal in menu order; with many items it may not find the best combination. Fine for typical orders.

Product and integration

GAP		DETAIL AND SUGGESTED NEXT STEP
-----	--	--------------------------------

High	Data handling not approved	Audio goes to Deepgram and transcripts to Gemini's free tier, where Google may use them for training. Next: Serv confirms data rules before any real customer audio is sent.
High	Recording start time is a guess	Without <code>AUDIO_START_UTC</code> or a filename timestamp, time comes from the file's modified time, which is unreliable. Matching orders to cars and payments depends on it. Next: confirm HME's metadata.
Medium	Batch, not streaming	The whole file is transcribed before the first order is extracted, so the first order waits for the entire file. Real-time would need streaming transcription and incremental segmentation.
Medium	No human-review loop	The schema defines <code>order.updated</code> and <code>order_version</code> , but nothing produces a version 2 yet. Next: a review screen that edits a <code>needs_review</code> order and re-sends it as <code>order.updated</code> .
Medium	Webhook schema not reviewed by Serv	Schema v1.0 adds <code>customer_declined_combo</code> , <code>unit_price</code> and <code>non_customer</code> to the draft. The payload also carries the full transcript, which Serv may or may not want (size and privacy).
Low	Single lane, single file	Dual-lane sites, lane bleed, and orders that span two files (flagged incomplete, not stitched) are out of scope.

Engineering and operations

	GAP	DETAIL AND SUGGESTED NEXT STEP
High	No authentication on the web app	Anyone who can reach the server can start runs (spending Deepgram credit and Gemini quota), upload files, read every payload in the mock inbox, and change its failure mode. Fine on localhost; must be fixed before it is reachable by anyone else.
Medium	Runs are lost on restart	The run queue lives in memory. If the server restarts, queued runs disappear and their rows stay "queued" or "running" forever. (Webhook deliveries are safe: the outbox recovers.) Next: on startup, re-queue or fail stale runs.
Medium	One run at a time; slow on long files	The queue is serial, and Gemini is paced at 10 requests per minute, so a recording with 60 cars needs at least 6 minutes of extraction. Next: a real job queue and a paid tier or higher limits.
Medium	Retries depend on a running process	Slow-phase retries only happen while <code>pnpm dev</code> or <code>pnpm pipeline worker</code> is running.
Medium	Single-machine infrastructure	SQLite, local files and an in-process worker suit a laptop. A deployment would use a managed database, object storage for audio, and a separate worker.
Low	No error monitoring	Failures are visible in the UI and logs only; there is no alerting (for example Sentry) for dead-lettered webhooks or failed runs.

12 How to explain it

IN 30 SECONDS

We take a drive-thru recording and produce one structured order per car, sent to Serv's server as a signed message. Speech-to-text writes down what was said. An AI model reads each conversation and lists what happened: "added a burger", "swapped Coke for Sprite", "asked about shakes". Then ordinary code, not the AI, turns that list into the order using the real menu and prices. Anything uncertain goes to a "needs review" list instead of being guessed. On 24 test recordings, it got every order right, using only free services.

IN 2 MINUTES (AN ANALOGY)

Think of three people working a drive-thru recording:

- A **stenographer** (Deepgram) writes down every word with a timestamp, and notes which words were hard to hear.
- A **note-taker** (Gemini) reads one conversation at a time and writes short notes: "customer added a large Coke", "customer changed the Coke to a Sprite", "crew said the total is \$7.88". The note-taker is not allowed to write the receipt.
- An **accountant** (our code) takes those notes, looks up the real menu and prices, applies them in order, and writes the receipt. The accountant checks the receipt against the total the crew said out loud, puts anything unclear in a "please check" pile, and keeps a record of everything that was cancelled or declined.

Then a **courier** (the webhook) delivers each receipt to Serv with a tamper-proof seal, and keeps trying until Serv confirms it arrived, without ever delivering the same receipt twice.

Questions people will ask

QUESTION	ANSWER
Why not have the AI write the order directly?	Because corrections, cancellations and prices are bookkeeping, and models are unreliable bookkeepers. Events keep the model's job small (what happened), and code does the part that must be exact. It also makes every result testable and explainable.
What happens when the AI is wrong or unsure?	Unknown items are caught by code and sent to <code>needs_review</code> with the top 3 likely menu items. Readback and total mismatches are flagged. Low audio quality is flagged. The system prefers "please check" over a confident wrong answer.

What if Gemini or Deepgram is down?	Calls retry with backoff. If Gemini still fails, a keyword matcher runs and everything goes to review, so the order still reaches Serv. Webhooks retry for about 28 hours before dead-lettering.
How do you know it is accurate?	A 24-fixture eval with known answers, covering all 28 edge cases from the brief, scored 100% with the real providers. The honest caveat is that the audio is synthetic and the same fixtures guided the tuning.
What does it cost?	Nothing so far. About 3,000 Gemini tokens per conversation and Deepgram per audio minute. Production would need a paid tier for volume and data privacy.
Can we switch providers?	Yes. Both sit behind interfaces; a new provider is one adapter file and a config value. The eval would show whether it is better.
How does Serv receive orders?	A signed HTTP POST to a URL they provide. They verify it with any Standard Webhooks library. For AWS (API Gateway plus Lambda), we change two settings.
Is it real-time?	Not yet. Each order is sent as soon as it is built, but only after the whole file is transcribed. Streaming is a known gap.
What is the biggest risk?	Real audio. Everything is proven on synthetic recordings. The first real HME files will show how far accuracy drops and which thresholds need calibration.

13 Open questions for Serv

Each answer replaces a placeholder in `config/sandbox.ts` or closes a gap in Section 11.

1. **Audio:** Do HME recordings have separate crew and customer channels? Which is which? What sample rate and format?
2. **Time:** How do we get the recording's real start time (metadata, filename, sidecar file)?
3. **Ground truth:** Can you share real recordings together with the orders the POS rang up, so we can measure accuracy and calibrate thresholds?
4. **Data rules:** May audio and transcripts go to third-party APIs? Is a no-training guarantee required (which rules out Gemini's free tier)?
5. **Menu:** Which brand and menu? Do you have a machine-readable menu with prices, combos and modifiers?
6. **Tax:** What tax rate applies per site, and do crews speak totals with tax?
7. **Webhook:** What URL and signing secret? Does the v1.0 schema work for you? Do you want the full transcript in each payload?
8. **Outcomes:** Do you want all statuses (completed, cancelled, abandoned, needs_review, incomplete), or only some?
9. **Latency:** Is "after the file is processed" acceptable, or do you need orders while the car is still at the speaker?
10. **Review:** Who resolves `needs_review` orders, and should corrected versions be re-sent as `order.updated`?
11. **Sites:** Site and lane IDs; any dual-lane locations?

A Commands, glossary, payload fields

Commands

```
pnpm install
pnpm test          107 unit and route tests, no API calls
pnpm lint          ESLint
pnpm typecheck     tsc --noEmit in every package
pnpm test:e2e      Playwright golden path (free providers)
pnpm dev           web app + mock webhook at http://localhost:3000
pnpm pipeline run <file.mp3> process a file and deliver its orders
pnpm eval          accuracy on every fixture, writes eval/report.json
pnpm eval --transcriber script --extractor oracle --no-judge the free ceiling
pnpm pipeline resend <order_id> resend a failed or dead-lettered delivery
pnpm pipeline worker slow-phase retries (the web app runs this for you)
pnpm pipeline examples regenerate examples/
pnpm pipeline settings list which Serv values are still placeholders
pnpm fixtures:build regenerate fixture audio (macOS)
```

Useful flags: `--transcriber deepgram|script`, `--extractor gemini|fuzzy|oracle`, `--channel-map 0=customer,1=crew`, `--start-utc <iso>`, `--no-deliver`, `--no-judge`, `--layout mono|stereo`, `--only 01_simple,04_correction`.

Glossary

HME	Maker of drive-thru headset systems; the source of the recordings.
ASR / STT	Automatic speech recognition / speech to text: audio in, words out.
Diarization	Telling speakers apart in one audio channel ("speaker 0, speaker 1"), without knowing who they are.
Utterance	One line of speech by one speaker, split at pauses.
Segment	One conversation, meaning one car's order.
Event	One thing that happened in a conversation (ADD, REPLACE, READBACK and so on).
Replay	Applying events in order to build the order, like applying transactions to a balance.
Keyterm	A word sent to Deepgram to make it more likely to recognize that word.
Webhook	An HTTP POST our system sends to Serv's server when something happens.
HMAC signature	A code computed from the message and a shared secret; proves the message came from us and was not changed.
Outbox	A table of messages waiting to be sent, written before sending so nothing is lost in a crash.
Dead letter	A message that failed every retry and now waits for a person.
Jitter	Random variation in retry timing so many senders do not retry at the same instant.
Fixture	A test recording with a known correct answer.

Oracle	An extractor that returns the known correct events; measures the ceiling of the rest of the pipeline.
SNR	Signal-to-noise ratio: how much louder speech is than the background, in decibels.

Webhook payload fields (schema v1.0)

schema_version , event_type	"1.0"; order.completed , .cancelled , .abandoned , .needs_review , .incomplete (and .updated , reserved)
order_id , order_version , group_id	Unique order ID; version (always 1 today); shared ID for split payments in one car
location_id , lane_id	Where it happened (placeholders today)
status , started_at , ended_at , timestamp_source	Outcome and wall-clock times, with how the start time was determined
audio	Source file and the order's start and end offsets in seconds
items , needs_review , not_ordered	The three buckets, each line with source utterance IDs and both confidences
combo_opportunities , customer_declined_combo	Missed meal deals with savings; whether the customer refused the meal
flags , totals , overall_confidence	Flags from Section 5.6; computed and spoken totals; average confidence
transcript	The conversation's lines, with speaker, times and confidence; crew chatter marked non_customer
processing	Speech and extraction providers, menu and pipeline versions, latency